

Performant Vectorized Python

A Technical Monograph on Writing Code That Saturates the Hardware

MARKDOWN · PARTS I–IX

[DOWNLOAD PDF](#)

Reconstruction notice. This page is a summary rebuilt from fragments of the original conversation. The linked PDF is a PDF render of *this reconstruction*, not the original monograph. To replace it with the full source, drop the real `.pdf` into `/pdfs/` using the same filename.

§ Abstract

Targeting CPython 3.13/3.14 and NumPy 2.x. Written for practitioners who already know Python and NumPy and want to understand the machine underneath. The thesis, threaded through every part: *performance work is memory work*. A double-precision multiply costs a few cycles; a fetch from main memory costs two to three hundred. Vectorized code is fast for two reasons that have different failure modes — it moves work out of the CPython interpreter, and it gives the compiler something the SIMD units can chew through. Treat those separately and the gap between typical numerical code and the hardware roofline collapses.

CONTENTS

- 01** Part I — The Machine (caches, vector units, the Roofline model)
- 02** Part II — NumPy Internals (`ndarray` memory, views vs copies, ufunc execution, broadcasting cost)
- 03** Part III — Writing Vectorized Code (`ArrayLike` + `asarray` , eliminating temporaries, AoS vs SoA, axis discipline)
- 04** Part IV — Reaching Hardware Performance (NumPy 2.x SIMD dispatch, cache blocking, Numba, dtype as a lever)
- 05** Part V — Parallelism (the GIL, free-threaded Python 3.13/3.14, the threading decision tree)
- 06** Part VI — Measurement (timeit → flame graphs, Roofline analysis, hardware counters)
- 07** Part VII — Beyond the CPU (GPU and accelerator paths; Mojo and the compiled-Python frontier)
- 08** Part VIII — The Frontier (NEP 54 + Google Highway, language changes, the convergence thesis)
- 09** Part IX — Cross-Language Perspective (Python vs C/C++/Java/Julia; a JAX field guide)
- 10** Annotated Bibliography (Drepper, Hennessy & Patterson, Roofline, Agner Fog, Rougier, NumPy NEP 38/54, Gorelick & Ozsvald)

§ The mental model: vectorization is really about memory

The instinct most people carry into performance work is wrong, or at least incomplete: they think performance means *doing fewer operations*. On modern hardware, the dominant cost of numerical code is almost never arithmetic. It is moving data. The arithmetic units sit idle, starved, waiting for operands.

This reframes everything. "Vectorized" code is fast for two distinct reasons, and they are worth separating because they have different failure modes:

1. **It moves work out of the CPython interpreter.** Each Python bytecode operation carries enormous overhead: boxed objects, reference counting, dynamic dispatch, type checks. A Python-level loop over a million floats pays that tax a million times; a NumPy call pays it once and then runs a tight C loop.
2. **It gives the compiler something the SIMD units can chew through.** Contiguous arrays in well-known dtypes are exactly what NumPy's inner loops — and now its runtime SIMD dispatch via NEP 38/54 — are tuned for.

§ API boundaries and dtype discipline

Accepting `ArrayLike` at the API boundary and normalizing internally is the right pattern:

```
from numpy.typing import ArrayLike, NDArray
import numpy as np

def propagate(state: ArrayLike, dt: ArrayLike) -> NDArray[np.float64]:
    state = np.asarray(state, dtype=np.float64)
    dt     = np.asarray(dt,    dtype=np.float64)
    ...
```

Use `np.asarray`, not `np.array` — `asarray` is a no-op when the input is already a contiguous array of the right dtype, so you avoid a spurious copy. Pinning `dtype` explicitly at the boundary prevents silent upcasts to `float64` from an unexpected `float32` input, or object-dtype disasters from a list of Python ints.

§ Layout: AoS vs SoA and the contiguous axis

Memory layout dominates cache behavior. Structure-of-arrays wins for vectorized math because each operation streams contiguous memory. For a batch of state vectors, store `(6,`

`N)` or `(N, 6)` deliberately. If your hot loop operates component-wise across the batch, you want each component contiguous: `positions[0]` should be a stride-1 view. Generally `(6, N)` with C-order gives you that; `(N, 6)` forces strided access on per-component ops.

Operate along the last axis. NumPy reduction and ufunc inner loops are fastest when the contiguous axis is the one being iterated.

§ Part VIII — the frontier

NEP 54 has progressed beyond draft. NumPy has begun using Google Highway internally and the roadmap states usage will expand, with SVE work ongoing. The crucial point for the user: it's internal re-plumbing with *no Python or C API changes*. You don't act on it; it silently widens free SIMD coverage — especially on ARM and future RISC-V. One practical wrinkle: Highway's pragma use is poorly supported by MSVC, so building NumPy from source on Windows with full SIMD may need `clang-cl`.

cuTile is genuinely new (December 2025, CUDA 13.1). Built on Numba plus a new compiler stack, draws on Triton, and introduces Tile IR as a virtual ISA. Worth prototyping to learn the model; Triton is the safer production pick today.

The §31 thesis: at the single-kernel level there is no meaningful performance ceiling separating compiled-Python tools from C. A well-shaped Numba/JAX/Cython kernel reaches the same hardware roofline as hand-written `-O3 -march=native` C. The cross-language benchmark literature backs this — Julia and C++ broadly comparable, small wins trading by scenario. The gap between typical numerical code and the maximum is almost never the language; it's the five fixable things: unfused temporaries, wrong layout, strided access, oversized dtype, un-exploited parallelism.



§ Status & provenance

Produced in conversation [71b47249](#) on 2026-05-17. The original markdown lives at [/mnt/user-data/outputs/performant-vectorized-python.md](#) in that container. Parts VIII–IX were the most time-sensitive — NEP statuses, version numbers, and tile DSLs will move; Parts I–IV are durable physics.